TECHADEMY

FORGE — Python for AI Scaffolding · Day 4 · 26-JUN-2026

Python for AI Scaffolding

Build the harness first — type-safe contracts, resilient clients, function-calling & FastAPI that keep AI systems stable under real-world pressure.

- pytest
- Pydantic v2
- typed LLM clients
- function-calling contracts
- FastAPI + OpenAPI
- hexagonal architecture
- async patterns

FOUNDATIONAL IDEA

What is “scaffolding”, and why start here?

Scaffolding is the supporting structure you put up around a building *while it is being constructed* — and the harness that keeps the workers safe. In software, scaffolding is the set of contracts, validation, error-handling, tests and layering you build **before and underneath** the actual AI features. It is what stops the whole thing collapsing when something goes wrong.

Covered in earlier sessions….

“Setting up admin and the scaffolding… understand scaffolding is the harness. Like building the ring on the scaffolding — you create a harness around it to hold it.” That is exactly the mindset: the harness goes up first.

The same idea, two worlds

Construction site

Scaffold + safety harness hold the structure and protect workers as floors go up.

AI software

Type hints + Pydantic + retries + tests hold the data and protect the system as features go up.

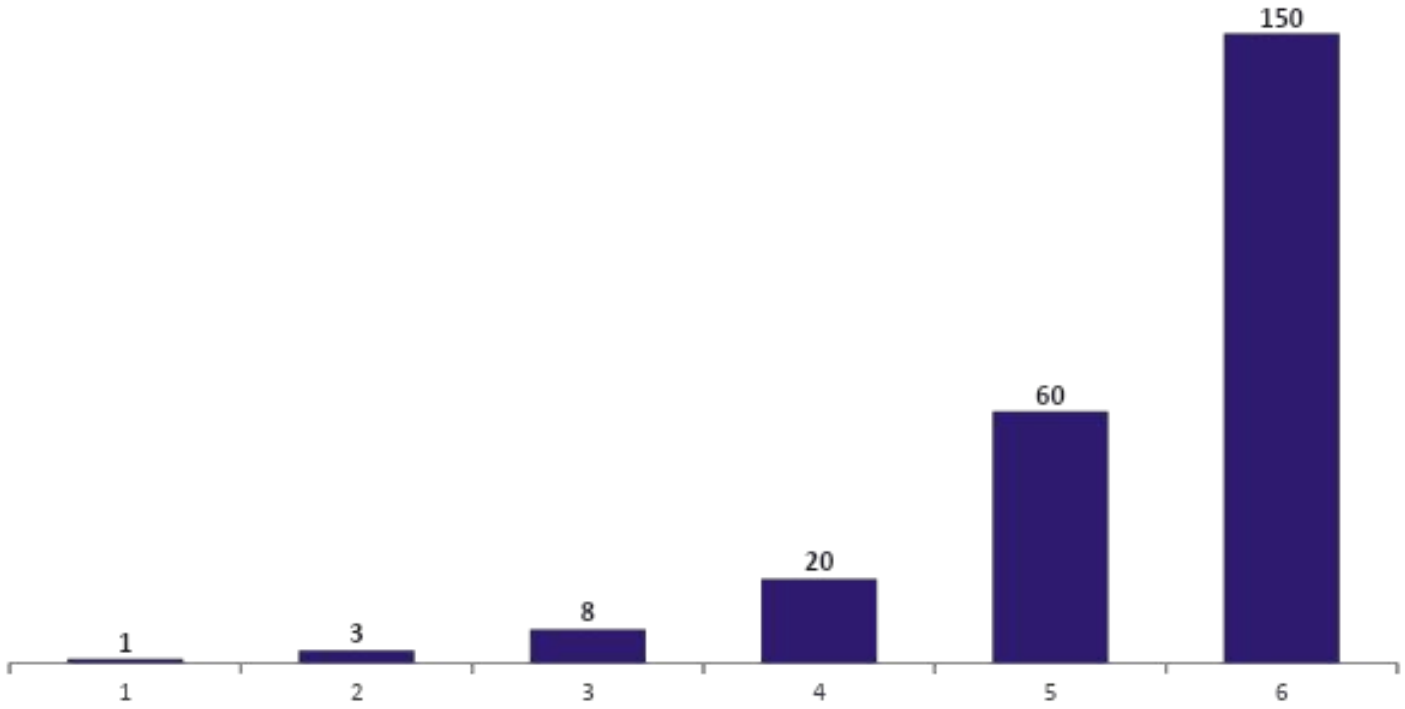
What both give you

Work can go fast WITHOUT fear of a sudden, expensive collapse.

THE BUSINESS CASE

Why scaffolding must start at the foundation

- The cost of fixing a defect grows sharply the later you catch it. Scaffolding pushes detection to the earliest, cheapest point — the moment bad data or a broken contract enters the system.



Where scaffolding earns its money

Every technique today moves the red bar leftward — catching the fault while it is still a “1”, not a “150”. That is the entire economic argument.

Real example from our work

In the offer-letter agent (Session 2), mistakes were being found only AFTER the letter reached the candidate — the most expensive possible point. Scaffolding the workflow with validation gates moves that catch to the very first step.

ROADMAP

What we will cover — six layers of the scaffold

01 Python for AI

Type hints · Pydantic v2 strict · async · packaging with uv

02 LLM SDK integration

Typed clients with retries — OpenAI · Anthropic · Azure OpenAI

03 Function-calling contracts

Schema definition · validation · error handling

04 REST APIs with FastAPI

Request/response models · OpenAPI specifications

05 Testing with pytest

Unit tests · contract testing for AI endpoints

06 Architecture primer

Separation of concerns · hexagonal · ports & adapters

HOW TO READ THIS SESSION

Every concept through four questions

To make sure nothing stays abstract, we examine each technique with the same four-question lens. Keep these four words in your head all day.

WHAT is it?

A plain-English definition, with every piece of jargon unpacked. No assumed knowledge.

WHY do it?

The risk you remove and the stability you gain. The failure it prevents.

HOW to do it?

The concrete pattern, in code, that you can copy into your own service today.

WHEN to do it?

The right moment to apply it — and, just as important, when it is overkill.

One promise for the whole day

Every technical term gets explained the first time it appears. If a word is new to you — async, idempotent, adapter, fixture — stop me. Nothing here is meant to be magic.

TECHADEMY

MODULE 01

Python for AI

Type hints · Pydantic v2 strict models · async patterns · packaging with uv

The language layer of the scaffold.

MODULE 1 · CONCEPT 1

Type hints — declaring the shape of your data

WHAT. A type hint is a small label on a variable, argument or return value that says what KIND of data it should be — `name: str` means "name is text". Python does not enforce it at runtime, but tools and libraries read these labels to check your work.

WHY. Three free wins: your editor autocompletes and flags mistakes as you type; libraries like FastAPI and Pydantic use the hints to validate and document automatically; and the next engineer reads the contract without guessing.

WHEN. Always, on every function boundary — especially anything that touches LLM output, request data, or another team's code. The boundary is where surprises enter.

HOW — label every boundary

```
def summarise(text: str, max_words: int) -> str:
    # editor now KNOWS the shapes,
    # and flags summarise(text, "ten")
    ...

# vs the silent-bug version:
def summarise(text, max_words):  # ?
```

Failure scenario — the silent dict

An LLM tool returns usage as a string "42". Untyped code does usage + 1 and crashes with a cryptic TypeError three layers deep at 2 a.m. With usage: int on the boundary, the wrong type is rejected exactly where it entered — named, located, instantly understood.

Pydantic v2 — turning type hints into a guard

Type hints describe; **Pydantic enforces.** A Pydantic model is a class where each field has a type and rules. When data arrives, Pydantic **validates and converts** it — or raises a precise error saying exactly which field was wrong and why.

WHAT

A data class that checks and cleans incoming data against typed rules at the moment it enters your code.

WHY

It is your single front door. Bad data is stopped at the gate, never spreading into the system as a hidden landmine.

HOW

Subclass BaseModel, add typed fields + Field() constraints, then parse raw input through it. v2 is Rust-fast.

WHEN

At every trust boundary: API requests, LLM/tool JSON, config files, queue messages, third-party responses.

v1 vs v2 — why “v2” matters

Pydantic v2 rewrote the core in Rust: 5–50× faster validation and a stricter, more predictable engine. For high-throughput AI endpoints validating every request, that speed is the difference between validation being free and being a bottleneck.

MODULE 1 · CONCEPT 2

Pydantic strict mode — refuse to guess

HOW — a strict model with rules

```python
from pydantic import BaseModel, Field

class User(BaseModel):
    model_config = {"strict": True}
    username: str = Field(min_length=3, max_length=20)
    age: int = Field(ge=18)

User(username="ok", age="21")
# ✗ rejected: too short; age must be int
```

Strict vs lax — the one setting that matters most for AI

Lax (default): Pydantic helpfully coerces — `"21"` becomes `21`. Convenient for humans, dangerous for AI: it hides the fact that the model returned the wrong type.

Strict: no silent guessing. A wrong type is a wrong type. You SEE the model misbehaving instead of inheriting a subtle bug downstream.

Failure — quiet coercion masks a bug

A pricing field comes back as "1,200" (with a comma). Lax mode silently turns it into 1200 in one provider and fails in another — inconsistent behaviour you can't reproduce. Strict mode rejects both, forcing you to clean the value once, explicitly.

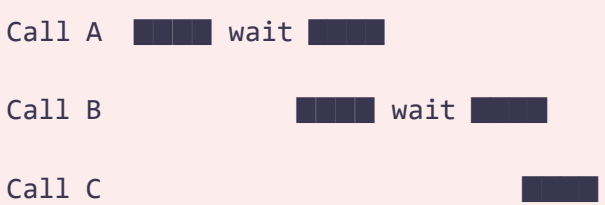
Protect — fail loud, fail early

Strict models turn "mysterious wrong number in the report" into "clear validation error at line 1". The system stays stable because malformed data never reaches business logic, the database, or the customer.

Async patterns — waiting without wasting

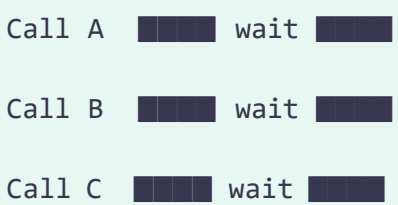
An LLM call spends almost all its time *waiting* for the network — often 2–20 seconds doing nothing. **Async** lets one worker start many such calls and handle whichever finishes first, instead of standing idle. It is about **overlapping the waiting**, not running faster.

Synchronous — one at a time



Total ≈ A + B + C (e.g. 18s)

Asynchronous — overlapped waiting



Total ≈ longest single call (e.g. 6s)

Jargon unpacked — concurrency ≠ parallelism

Concurrency = dealing with many things by switching between them while they wait (one cook, many pots). Parallelism = doing many things at literally the same instant (many cooks). Async gives you concurrency on a single worker — perfect for I/O-bound work like LLM and database calls, where you are mostly waiting.

MODULE 1 · CONCEPT 3

Async — how to use it, and the trap to avoid

HOW — fire many LLM calls, gather results

```python
import asyncio

async def summarise(doc):
    return await client.messages.create(...)

# run 50 summaries with overlapped waiting
results = await asyncio.gather(
    *[summarise(d) for d in docs]
)
```

WHEN to reach for async

I/O-bound work: LLM calls, HTTP, database, file/queue reads. NOT for CPU-bound number-crunching — that needs multiprocessing, not async.

Two words you must know

`async def` marks a function that can pause. **`await`** is the pause point — "let others run while I wait here".

Failure scenario — blocking the event loop

An engineer drops a slow synchronous call — time.sleep(5), a heavy pandas computation, or requests.get — inside an async endpoint. That ONE blocking line freezes the single worker; every other in-flight request stalls behind it and the whole service appears to hang. Rule: inside async code, never call blocking functions — use the async version, or push the heavy work to a thread/process pool.

MODULE 1 · CONCEPT 4

Packaging with uv — “works on my machine”, solved

WHAT

uv is a fast, modern Python package & environment manager — one tool replacing pip, venv, and more, with an exact lockfile.

WHY

It pins EVERY dependency to an exact version, so your laptop, your teammate, CI and production all run identical code. No drift.

HOW

uv init · uv add anthropic fastapi · uv run app.py. A uv.lock file captures the full, reproducible tree.

WHEN

From the very first commit of every project. Reproducibility is cheapest when established on day one, costly to retrofit.

Failure — the dependency that changed under you

Production worked Friday, broke Monday. Nobody changed code — but a sub-dependency released a new version overnight, and the unpinned install pulled it in. Hours lost to a bug you didn’t write. A lockfile makes installs byte-for-byte repeatable and immune to this.

Protect — a reproducible foundation

With uv’s lockfile, “works on my machine” becomes “works on every machine”. Onboarding a new engineer or rebuilding a server is one command. This is scaffolding for the environment itself — the floor the whole app stands on.

MODULE 1 · RECAP

The language layer of your scaffold

Type hints	Label every boundary so tools catch wrong shapes before they spread.
Pydantic v2 strict	One front door that validates and refuses to silently coerce bad data.
Async patterns	Overlap the waiting on I/O-bound LLM calls; never block the event loop.
Packaging with uv	A lockfile makes every machine identical — reproducibility from day one.

One sentence: write down what your data should look like, and let the tools enforce it — everywhere, from day one.

TECHADEMY

MODULE 02

LLM SDK Integration

Typed clients with retries — OpenAI · Anthropic · Azure OpenAI

Talking to LLMs without your service falling over.

LLM calls fail — by their nature, not by accident

A call to an LLM leaves your building and crosses the public internet to someone else's busy servers. Treat every call as something that WILL sometimes fail. Scaffolding here means planning for failure, not hoping it away.

Rate limits (429)

"Too many requests." The provider throttles you during a traffic spike — exactly when you need it most.

Timeouts

The model takes 40s on a hard prompt; your 10s timeout fires. The work isn't wrong — it's just slow.

Transient 5xx

The provider has a brief blip. The same request succeeds if you simply try again a second later.

Malformed output

You asked for JSON; you got JSON wrapped in an apology. Parsing explodes.

Auth / config errors

Wrong key, missing region, model not enabled. Common on first deploy — "AWS not configured".

Network drops

DNS hiccup, connection reset. The internet is not a reliable wire.

The typed client — one wrapper, not scattered calls

Don’t sprinkle raw `client.messages.create(...)` across forty files. Wrap the SDK in ONE typed client class with a typed input and a **Pydantic-validated output.** Now every LLM call in the system flows through a single, controllable choke-point.

HOW — wrap the SDK behind a typed method

```
class Summary(BaseModel):
    text: str;  tokens: int

class LLMClient:
    async def summarise(self, doc: str) -> Summary:
        raw = await self._sdk.create(...)
        # validate BEFORE returning to callers
        return Summary.model_validate_json(raw)
```

WHY this one choke-point pays off

Add a retry, swap a provider, change a timeout, add logging or cost-tracking — you edit ONE class, and the entire app benefits. No hunting through forty files.

Protect — a controllable seam

The typed client is the seam where all resilience lives. Callers receive a clean, validated Summary object and never touch raw SDK responses. The mess is contained in one place by design.

MODULE 2 · CONCEPT 2

Retries — surviving transient failure gracefully

WHAT

Automatically re-attempting a failed call that is likely temporary — a 429, a timeout, a brief 5xx — instead of giving up immediately.

WHY

Most LLM failures are transient. A quiet, well-timed retry turns a user-visible error into a small, invisible delay.

HOW

Exponential backoff + jitter: wait 1s, 2s, 4 s … plus a small random nudge, for a capped number of attempts.

WHEN

Only for transient + idempotent operations. NEVER blindly retry a 400 bad-request or a non-idempotent side-effect.

Jargon — backoff, jitter, idempotent

Backoff: wait longer after each failure, to let the provider recover. Jitter: add randomness so a thousand clients don't all retry at the same instant (a "thundering herd"). Idempotent: safe to repeat — doing it twice has the same effect as once.

Failure — the retry storm

A provider blips; your app retries instantly, with no backoff, from every worker at once. The flood of retries hammers the recovering provider and your own system — turning a 2-second blip into a 20-minute outage you caused. Backoff + jitter is what prevents this.

Retries — the pattern, and when to STOP

HOW — tenacity: backoff + jitter, capped

```python
from tenacity import retry, wait_exponential_jitter

@retry(
  stop=stop_after_attempt(4),
  wait=wait_exponential_jitter(max=20),
  retry=retry_if_exception_type(Transient)
)
async def call_llm(...): ...
```

WHEN to retry — and when NOT to

- Retry: 429, 503, timeouts, connection resets
- Retry: read-only or idempotent operations
- Don’t: 400 / 422 — your request is wrong; retry won’t help
- Don’t: 401 / 403 — auth/config; fix the key, not the retry
- Don’t: non-idempotent side-effects without an idempotency key

The full resilience recipe (sits inside your typed client)

timeout (don’t wait forever) → retry transient errors with backoff + jitter (capped attempts) → circuit-breaker (if the provider is clearly down, stop hammering and fail fast) → fallback (use a cheaper model, a cached answer, or a graceful “try later”). Layered like this, a provider outage degrades your service instead of crashing it.

MODULE 2 · CONCEPT 3

One interface, three providers — the adapter

OpenAI, Anthropic and Azure OpenAI each have their own SDK and quirks. Define your OWN small interface — *"give a prompt, get a validated Summary"* — and write a thin **adapter** for each provider behind it. Your app depends on YOUR interface, never on a vendor's SDK directly.

WHY — vendor independence is leverage

Switch providers for price, latency, compliance or an outage — by writing one new adapter, with zero changes to business logic. You also dodge vendor lock-in, the exact concern raised in our Session 2 discussion of Claude SDK lock-in vs framework neutrality.

WHEN — and a note on Azure OpenAI

Build the interface from the start even with one provider — it costs little and saves a painful refactor later. Azure OpenAI is the same OpenAI models but deployed inside your Azure tenant for enterprise compliance/data-residency — usually an adapter difference of endpoint, auth and deployment name, not logic.

From our work — the guardrail client that handles its errors

In the JD-analysis system (Session 1), an AWS Bedrock guardrail agent screened model input and output. What made it production-grade was not the happy path — it was the explicit error handling around every way the call could fail.

Guardrail did not intervene	Pass the content through — return passed = true.
Guardrail intervened	Block the content and return a clear, structured “intervened” message with the reason.
AWS not configured / key missing	Caught explicitly — surfaced as a precise config error, not a mysterious crash.
AWS / Bedrock service error	Caught and reported with the blocked-reason detail, so on-call knows exactly what broke.

The lesson: a real LLM client spends as much code on what-goes-wrong as on what-goes-right. That balance IS the scaffolding.

MODULE 2 · RECAP

Talking to LLMs, Safely

Assume failure	Rate limits, timeouts, blips and bad output are normal — design for them.
Typed client wrapper	One choke-point where validation and resilience live; clean objects out.
Retries done right	Backoff + jitter, capped, only for transient + idempotent calls.
Adapter per provider	Depend on your interface, not a vendor SDK — switch freely, avoid lock-in.

One sentence: route every LLM call through one typed, resilient client so failure becomes a delay, not an outage.

TECHADEMY

MODULE 03

Function-Calling Contracts

Schema definition · validation · error handling

When the LLM calls your code, the contract is everything.

What is function calling — and why it needs a contract

Function calling lets the LLM decide to run one of YOUR functions and supply the arguments. You hand it a menu of tools (each with a name and an argument *schema*); it replies with "call `create_ticket` with these arguments". The schema IS the contract between a probabilistic model and your deterministic code.

The core risk in one line

The model is PROBABILISTIC — it usually produces correct arguments, but it can hallucinate a field, send the wrong type, invent an enum value, or return half-formed JSON. Your code is DETERMINISTIC and will faithfully execute whatever it's given. The contract — schema + validation — is the seatbelt between the two.

MODULE 3 · STEP 1

Schema definition — describe the contract precisely

HOW — the tool schema is a Pydantic model

```
from enum import Enum
class Priority(str, Enum):
    low="low";  high="high"

class CreateTicket(BaseModel):
    title: str = Field(max_length=120)
    priority: Priority
    assignee_id: int

# hand model_json_schema() to the LLM as the tool
```

WHY define it tightly

A precise schema constrains the model's creativity to a safe surface. Enums stop invented values; max_length stops runaway strings; required fields stop silent omissions. You are narrowing the blast radius before the model even replies.

Case — the JD-analysis agents

Each specialist agent (skill, completeness, cognitive-load, inclusion, compensation) was told: "return ONLY valid JSON matching this exact schema — no markdown, no explanation." The schema was the guardrail that made five independent agents composable.

WHEN — and a discipline

Define the schema BEFORE you write the function body, and keep it the single source of truth: the same Pydantic model generates the tool definition for the LLM, validates the response, AND documents the contract. One declaration, three jobs — the same collapse-of-boilerplate idea you saw with FastAPI typing.

Validation — never trust the arguments blindly

The model returned arguments. They are a PROPOSAL. Before a single line of your function runs, parse those arguments through the schema. If they don't fit, you reject — you do not execute.

HOW — validate at the door of execution

```
raw_args = tool_call.arguments    # from the LLM
try:
    args = CreateTicket.model_validate(raw_args)
except ValidationError as e:
    # DON'T run. Return the error to the model
    return tool_error(e.errors())  # self-correct

create_ticket(**args.model_dump())  # safe now
```

WHY return the error TO the model

Modern function-calling loops can self-correct: feed the validation error back and the model usually fixes its own arguments on the next turn. Validation becomes a conversation, not a dead end.

WHEN — every call, no exceptions

There is no "trusted" tool call. Even a 99%-reliable model, over a million calls, sends ten thousand bad ones. Validate 100% of the time.

Protect — the boundary that holds

Validation is the wall between "the model said so" and "my database changed". With it, a hallucinated argument becomes a caught, logged, recoverable event. Without it, that same hallucination becomes a corrupted record, a wrong assignee, or a deleted row — silently.

Error handling — plan for the model misbehaving

Half-formed JSON

Wrap parsing in try/except; return a structured “invalid JSON” error for the model to retry.

Hallucinated field

Strict schema rejects unknown keys (extra="forbid"); the model is told the field doesn’t exist.

Wrong enum / type

Validation names the exact field; feed it back so the model corrects, e.g. low/high not “urgent-ish”.

Dangerous-but-valid args

Schema-valid ≠ business-valid. Add a logic check: e.g. can THIS user be assigned? Authorise after validating.

Failure scenario — the schema-valid disaster

A support agent tool exposes delete_records(filter). The model, mis-reading a vague request, calls it with an EMPTY filter — perfectly valid against the schema, so validation passes — and the function deletes every row. Lesson: validation checks SHAPE; a separate business-rule + authorisation check must guard INTENT. Never expose a destructive, unbounded tool without both.

MODULE 3 · RECAP

Contracts between a probabilistic model and your code

Schema = the contract	A tight Pydantic model (enums, limits, required) narrows the model's blast radius.
Validate every call	Arguments are a proposal; parse before you execute, 100% of the time.
Return errors to the model	Validation failures feed back so the model self-corrects — a loop, not a wall.
Guard intent, not just shape	Add business-rule + authorisation checks; schema-valid ≠ safe to run.

One sentence: treat every tool call as untrusted input — validate its shape, then check its intent, before your code acts.

TECHADEMY

MODULE 04

REST APIs with FastAPI

Request/response models · OpenAPI specifications

Exposing the scaffold to the world — safely and clearly.

FastAPI — your type hints become the API contract

REST API = the doorway other programs use to talk to your service over HTTP (GET to read, POST to create, etc.). **FastAPI** reads your Python type hints and Pydantic models and, from them alone, validates every request, serialises every response, and generates live documentation. Everything you built in Module 1 now pays off at the edge.

WHAT

A modern Python web framework that turns typed functions into HTTP endpoints, with validation and docs for free.

WHY

Zero-boilerplate validation: the same Pydantic model guards the request, shapes the response, and documents the API.

HOW

Decorate a typed async function with @app.post; declare request and response models; FastAPI does the rest.

WHEN

Default choice for AI service backends — async-native, fast, and it reuses the typing scaffold you already have.

Why it became the standard for AI backends

Before FastAPI you chose fast-but-manual (Flask) or robust-but-heavy (Django). FastAPI used type hints to automate the tedious, error-prone parts — and being async-native, it suits LLM workloads where you're mostly waiting on the network.

MODULE 4 · CONCEPT 2

Request & response models — typed doors, both ways

HOW — declare both directions explicitly

```python
class AskRequest(BaseModel):
    question: str = Field(min_length=1)

class AskResponse(BaseModel):
    answer: str
    sources: list[str]
    # NOTE: no internal_cost, no raw_prompt here

@app.post("/ask", response_model=AskResponse)
async def ask(body: AskRequest) -> AskResponse: ...
```

WHY a response model, not just a request one

The request model keeps bad data OUT. The response model keeps secret data IN — it whitelists exactly which fields leave your service, so internal cost, prompts or PII can never leak by accident.

WHEN — always declare both

Returning a raw dict “just this once” is how internal fields escape. Make the response_model mandatory in code review. It is the cheapest data-leak prevention you will ever add — one line per endpoint.

Protect — the edge is your perimeter

Two Pydantic models turn an endpoint into a controlled airlock: validated data in, whitelisted data out. Combined with the typed LLM client inside, a request now passes through layers of checks before anything irreversible happens — and nothing unexpected ever flows back to the caller.

OpenAPI specs — the contract everyone can read

An **OpenAPI spec** is a machine-readable description of your whole API — every endpoint, its inputs, outputs and errors. FastAPI generates it **automatically** from your models, and serves live docs at `/docs`. The spec is a single source of truth your frontend, your clients, your tests and your partners all share.

Always-current docs

Generated from code, so the docs can never drift out of date the way hand-written ones do.

Client code generation

Frontend and partner teams auto-generate typed clients from the spec — no manual guesswork.

Contract for tests

Tests assert responses match the spec, catching breaking changes before release (Module 5).

Shared language

One artefact aligns backend, frontend, QA and external integrators on the exact same contract.

MODULE 4 · SIDE-BY-SIDE

Same task, two approaches — what typing buys you

Endpoint: POST /users requiring username (3–20 chars) and age (≥ 18); invalid data returns a clear 400.

Flask — manual, defensive validation

```
data = request.get_json()
errors = {}
if not isinstance(data.get("username"),str):
    errors["username"]="must be str"
# …~30 more lines of hand checks…
if errors: return jsonify(errors), 400
```

FastAPI — type-driven, automatic

```
class UserSchema(BaseModel):
  username: str = Field(min_length=3,max_length=20)
  age: int = Field(ge=18)

@app.post("/users", status_code=201)
async def create(user: UserSchema):
  return user  # already validated & parsed
```

Dimension	Flask	FastAPI
Validation code	30+ defensive lines	0 — from the type hints
Error output	Whatever you hand-write	Structured JSON naming the exact field
Add a field	Write a new if/else block	Add one typed line to the model
Docs	Separate, often stale	Auto-generated, always current

MODULE 4 · FAILURE & DECISION

A leak you won’t see — and where logic should live

Failure scenario — the response without a model

An /ask endpoint returns the raw internal object “to save time”. It happens to include raw_prompt, internal_cost_usd and a debug user_email. None show in the UI — so nobody notices for months — until a partner inspects the JSON and finds your prompts and a customer’s email exposed. A declared response_model would have stripped all three automatically.

Architecture decision from our work — where does business logic belong?

Keep logic in the BaaS layer (e.g. Supabase Edge)	Move logic into a FastAPI service
App will always run on that platform	App must be portable — client may bring their own Postgres/Oracle
Fewer moving parts, less DevOps	Database layer becomes interchangeable
Platform handles auth for you	You own auth — JWT, sessions — but you control it
Best for: SaaS you control end-to-end	Best for: enterprise clients who dictate infrastructure

“Where business logic lives is a business decision, not a technical one.” — a solution-architect call, made before you write the endpoint.

MODULE 4 · RECAP

Exposing the scaffold as an API

Typing pays off at the edge	FastAPI turns your hints into validation, serialisation and docs — for free.
Two typed doors	Request model keeps bad data out; response model keeps secret data in.
OpenAPI as source of truth	Auto-generated spec aligns frontend, clients, tests and partners.
Decide where logic lives	BaaS vs FastAPI is a business/portability decision — make it deliberately.

One sentence: let your types BE the public contract — validated in, whitelisted out, documented automatically.

TECHADEMY

MODULE 05

Testing & Contract Testing

Proving the scaffold actually holds — automatically, on every change, even with a non-deterministic model in the middle.

When the answer changes every time, you test the SHAPE — not the words.

pytest — your automated safety net

pytest is the standard Python testing tool. You write small functions that call your code with known inputs and **assert** what should come back. Run one command and every check runs in seconds — so a change that quietly breaks something is caught *before* it reaches a user.

Why it matters for an AI system

Change without fear

Refactor a prompt or swap a provider and instantly know nothing else broke.

Fast feedback

Seconds, not a manual click-through. The harness checks itself.

Living documentation

A good test reads as a spec: 'given this input, the contract says this output'.

test_pricing.py — a plain pytest test

```
from app import quote_price

def test_quote_adds_gst():
    price = quote_price(100)
    # 18% GST expected
    assert price == 118

def test_rejects_negative():
    with pytest.raises(ValueError):
        quote_price(-5)

$ pytest -q
2 passed in 0.04s
```

MODULE 5 · HOW

Three habits — and the golden rule: never call a real LLM in a test

Fixtures

Reusable setup (a test client, a fake DB) declared once and injected into many tests. No copy-paste.

Parametrize

Run the same test over many inputs with one decorator — 10 edge cases, one function.

Mock the LLM

Replace the model call with a canned response. Tests become fast, free and repeatable.

Mock the model — deterministic, offline, free

```
def test_summary_is_short(mocker):
    # pretend the LLM replied with this
    mocker.patch("app.llm.call",
        return_value="a short summary")
    out = summarise("...long text...")
    assert len(out.split()) < 20
```

Why mocking is non-negotiable

A real API call is slow, costs money, needs a key, and returns a DIFFERENT answer each run — so the test would randomly fail. Mocking makes the test deterministic. You test YOUR logic (does it handle the reply correctly?), not the model's mood today.

MODULE 5 · THE AI-SPECIFIC PROBLEM

Contract testing — when the output is never the same twice

The non-determinism trap

You write assert reply == "The total is ₹118.". It passes today. Tomorrow the model says "Your total comes to ₹118." — same meaning, test fails. Assert on exact AI wording and your test suite becomes a wall of false alarms everyone learns to ignore. That is worse than no tests.

The fix: stop testing the words. Test the CONTRACT — the things that must always be true.

Test the shape

Is it valid JSON? Does it parse into the Pydantic schema? Are required fields present and typed?

Test the properties

Is the summary under 50 words? Is the sentiment one of the allowed enums? Is the total a positive number?

Golden + eval tests

Keep a set of known inputs with expected PROPERTIES; an eval gate scores model output on a rubric, not a string match.

Wire it into the gate — catch drift before the frontend does

Contract test against the Pydantic schema

```python
def test_analysis_obeys_contract():
    raw = run_real_model(SAMPLE_JD)
    # must parse into the agreed schema
    result = JDAnalysis.model_validate_json(raw)
    # properties that must always hold
    assert 0 <= result.score <= 100
    assert result.bias_flags is not None
    assert len(result.summary) < 500
```

When does it run? On every pull request, as a **CI gate** — the same review-gate discipline from our spec-driven sessions. Code cannot merge until the contract holds. Run eval gates nightly on a larger sample.

From our JD-analysis system

Each specialist agent was told to return ONLY valid JSON matching an exact schema. A contract test that model_validate_json's every agent's output is the automated version of that rule — drift is caught in CI, not in production.

The drift failure this prevents

A model upgrade subtly renames a field: "full_name" becomes "name". Runtime still 'works', but the frontend reading full_name now shows blanks to every user. A contract test asserting the schema would have gone red the moment the field moved — turning a silent production outage into a failed CI check nobody even merged.

MODULE 5 · RECAP

Tests are the part of the scaffold that checks itself

pytest is the net	Small assert-driven tests run in seconds and catch breakage before users do.
Mock the model in unit tests	Deterministic, fast, free — test YOUR logic, not the model's mood.
Assert the contract, not the words	Shape, properties, schema — never exact AI wording, or the suite becomes noise.
Make it a gate	Contract tests in CI turn silent drift into a red build that can't merge.

One sentence: if you can't test it automatically, you can't change it safely — so test the contract and gate every merge.

TECHADEMY

MODULE 06

Architecture Primer

Separation of concerns · hexagonal layering · ports & adapters

Arrange the pieces so any one of them can be replaced without touching the rest.

Separation of concerns — one job per part

Separation of concerns means each part of your system does **one job** and knows as little as possible about the others. Your business rules should not know whether the data came from OpenAI or Azure, from Postgres or Supabase, or arrived over HTTP or a queue. Mix those concerns together and one small change ripples everywhere.

The tangle this avoids

An endpoint function that parses the HTTP body, calls OpenAI directly, runs the business logic, AND writes raw SQL — all in 200 lines. Swap the LLM provider and you are editing the same function that handles HTTP and SQL. Every change risks breaking three unrelated things at once.

One request, three separated jobs

Edge layer

FastAPI route: validate the request, shape the response. Knows HTTP — nothing else.

Core / domain

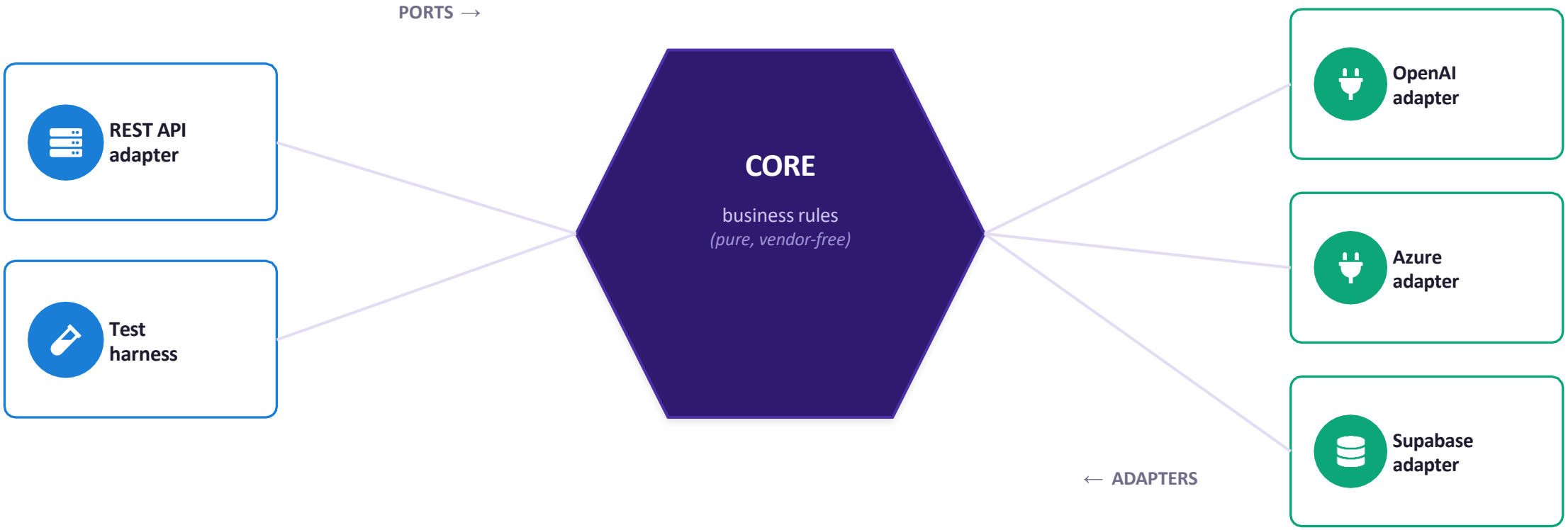
The business rules. Pure Python. Knows NO vendor, NO database, NO framework.

Infrastructure

LLM clients, DB drivers, queues. Knows vendors — but not your business rules.

Hexagonal architecture — ports & adapters

A **port** is an interface your core defines ("I need something that can summarise text"). An **adapter** is a concrete implementation that plugs into it (the OpenAI adapter, the Azure adapter). The core depends only on the port — never on the adapter.



The core talks to the OUTSIDE only through ports. Swap an adapter and the core never notices.

MODULE 6 · WHY & HOW

The payoff: swap any adapter, never touch the core

WHY

Vendors change, prices change, clients dictate infra. If switching providers means a rewrite, you are trapped. Ports make it a one-file change.

HOW

Define a port as a Protocol/ABC. Write one adapter per vendor. Inject the chosen adapter at startup. The core only ever sees the port.

RESULT

OpenAI → Azure, Postgres → Supabase, real → mock: all become config, not surgery. Tests inject a fake adapter for free.

WHEN

Apply ports at every external boundary from day one. Retro-fitting later means untangling the very mess this prevents.

One port, many adapters — the core depends only on the Protocol

```
class Summariser(Protocol):        # the PORT (core owns this)
    def summarise(self, text: str) -> str: ...

class OpenAIAdapter:  ...    class AzureAdapter:  ...    # ADAPTERS plug in
core = Engine(summariser=pick_adapter(settings))  # chosen at startup
```

MODULE 6 · FAILURE

The “big ball of mud” — what happens with no layering

Failure scenario — everything touches everything

A team ships fast with no separation. Eighteen months on, the OpenAI SDK is imported in 40 files, SQL is scattered through route handlers, and business rules live inside HTTP code. Then the client mandates Azure OpenAI and an on-prem database. What should be a config change becomes a three-month rewrite touching 40 files — each one a chance to introduce a new bug. The team stops shipping features and just fights the tangle.

The discipline that would have prevented it — applied from day one:

Core stays pure

No SDK imports, no SQL, no framework in the business layer — ever.

Vendors behind ports

Every external system reached only through an interface the core owns.

Inject at the edge

Choose the adapter once, at startup. Everywhere else, code to the port.

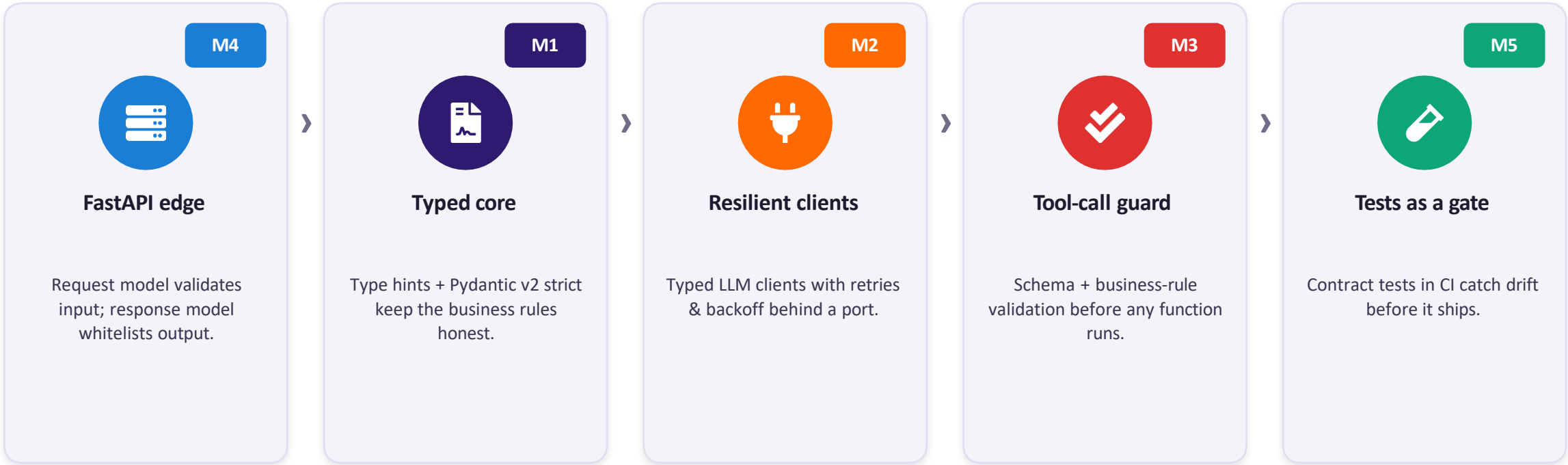
Tests prove the seams

If you can inject a mock, your boundaries are clean. If you can't, they're tangled.

PUTTING IT TOGETHER

The full scaffold — all six modules on one picture

A request enters from the left, passes through every layer of the harness, and only then touches anything irreversible. Each module you learned today is one ring of protection.



M6

Architecture holds it all — the frame that keeps every layer independent and swappable.

TAKE IT BACK TO YOUR DESK

The scaffolding checklist — audit any AI service against this

- Every function boundary has type hints
- Tool-call arguments are validated before execution
- LLM output parses into a Pydantic v2 strict model
- Destructive tools have business-rule + auth guards
- I/O is async; the event loop is never blocked
- FastAPI request AND response models are declared
- Each LLM client is typed, with retries + backoff
- Contract tests run as a CI gate on every PR
- Provider sits behind a port, not imported in the core
- The core imports no SDK, no SQL, no framework

If you can tick all ten, the harness is up — your team (and an AI agent) can build fast without fear of a collapse.

TECHADEMY

The one idea to keep

Put up the harness before you build the floors.

Type-safe contracts, resilient clients, validated tool-calls, a typed API, gated tests and clean layering aren't six chores — they're one practice. Build the scaffold from the foundation and your AI system stays standing when the real world pushes on it.

Questions — and let's map it to YOUR current project.